Sharding & Consistent Hashing

Lesson 13 · splitting data across machines

In this lesson → ① Why shard ② Shard key ③ hash % N ④ The ring ⑤ Virtual nodes ⑥ Hot partitions ⑦ Costs ⑧ Interview Q&A

Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

> **Sharding** (partitioning) = splitting data across machines so each **shard** stores and serves only part of it. Replication scales **reads**; sharding scales writes and storage.

1 Shard key + strategies

Strategy	How	Good	Watch out
Range	A–H, I–P, Q–Z	range queries	hot ranges
Hash	hash(key) → shard	even spread	range queries hit all shards
Directory	lookup service	flexible moves	extra hop; must be HA

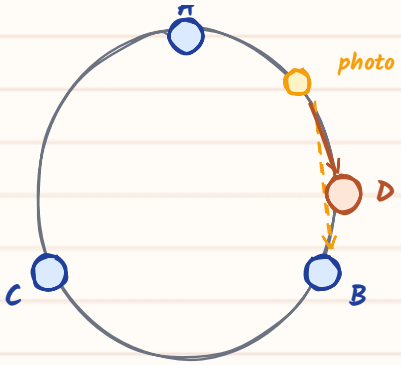
2 Why hash % N breaks

	hash(key) % 4	hash(key) % 5
key 17	→ server 1	→ server 2 moved
key 42	→ server 2	→ server 2 ✓
key 58	→ server 2	→ server 3 moved
key 91	→ server 3	→ server 1 moved
key 100	→ server 0	→ server 0 ✓
key 123	→ server 3	→ server 3 ✓

add one server → most keys move

SYSTEM DESIGN NOTEBOOK

3 Consistent hashing



hash(server) → point on ring

hash(key) → walk clockwise

add D → only keys between A and D move

≈ 1/N of keys, not 80%

- → Hash servers and keys onto the same ring
- → Key → next server **clockwise**
- → Add/remove a node → only **~1/N** of keys move

4 Virtual nodes

- ✓ Each server placed at ~100+ points
- ✓ Even spread; a leaving node's load is shared
- ✓ Weighted: bigger machine → more vnodes
- ✓ Used by Cassandra, DynamoDB, Riak

5 Hot partitions (celebrity problem)

✗ One key gets huge traffic → one shard melts

✓ **Split** the key: post_id#0...9, sum on read

✓ **Cache** hot items in front

✓ **Isolate**: dedicated shard for giants

6 What sharding costs

✗ Cross-shard joins and transactions

✗ Queries without the shard key → scatter-gather

✗ Rebalancing, resharding, operations

SnapLink

Shard photos by **user_id** → a profile page reads one shard. Shard as late as you can; tools: Vitess (MySQL), Citus (Postgres).

7 Interview Corner

Q. When do you shard?

A. When data or write volume outgrows one machine (replicas only scale reads).

Q. How do you choose a shard key?

A. High cardinality, even spread, and most queries include it (one shard per query).

Q. Why not hash(key) % N?

A. Changing N remaps most keys, forcing massive data movement.

Q. What do virtual nodes solve?

A. Uneven arcs on the ring; they spread data and failover load evenly.

Q. How do you handle a hot key?

A. Split it with suffixes, cache it, or give it a dedicated shard.

⚡ Revise in 60 seconds

- → Sharding scales **writes**
- → Pick the **shard key**
- → Range vs hash vs directory
- → % N moves most keys
- → **Ring**: next server clockwise
- → **~1/N** keys move
- → **vnodes** even it out
- → Hot key: split / cache

Next lesson → SD-14
Transactions and isolation levels.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

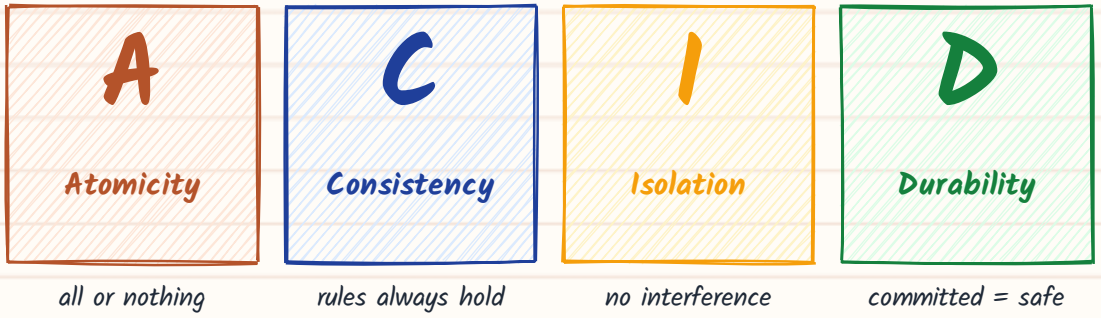
Transactions & Isolation Levels

Lesson 14 · ACID, anomalies, locking & double booking

In this lesson → ① Crash in the middle ② ACID ③ Under the hood ④ Anomalies ⑤ Isolation levels ⑥ Double booking ⑦ Locking ⑧ Across services ⑨ Interview Q&A
Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

> A **transaction** groups several reads and writes into one unit: all succeed, or none do.

1 ACID



Letter	Means	How
A	all or nothing	WAL + rollback
C	constraints / rules hold	your schema + app logic
I	concurrent txns don't interfere	locks or MVCC
D	committed survives a crash	WAL flushed to disk

2 Under the hood

→ **WAL**: log first → after crash replay committed, undo the rest

→ **MVCC**: each txn reads a snapshot; readers don't block writers

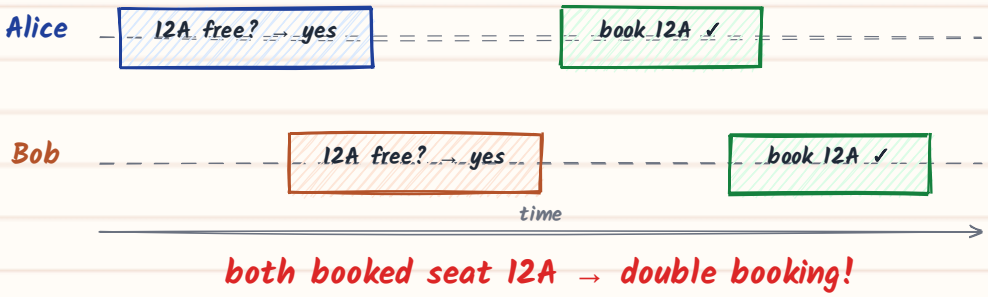
3 Anomalies

Anomaly	What happens
Dirty read	see uncommitted data that may roll back
Non-repeatable read	same row twice → different values
Phantom read	same query twice → new rows
Lost update	two read-modify-writes → one lost
Write skew	two txns each valid, together break a rule

4 Isolation levels

Level	Dirty	Non-repeatable	Phantom
Read uncommitted	✗	✗	✗
Read committed (Postgres)	✓	✗	✗
Repeatable read (MySQL)	✓	✓	~ (snapshot helps)
Serializable	✓	✓	✓

5 Double booking (write skew)



6 Pessimistic vs optimistic

```
SELECT * FROM seats WHERE id = '12A' FOR UPDATE;  -- pessimistic

UPDATE seats SET owner = 'bob', version = 6
 WHERE id = '12A' AND version = 5;       -- optimistic (0 rows → retry)
```

- ✓ Pessimistic: high contention (tickets, inventory)
- ✓ Optimistic: low contention (profile edits)
- ✓ Unique constraint (flight, seat) as a safety net

7 Across services

✗ 2PC: slow, blocking coordinator

✓ **Saga**: local transactions + compensating actions

✓ **Outbox**: reliable events with the local write

SnapLink
Pro purchase = one Postgres transaction (wallet + plan + order). Payment provider call → saga with refund as compensation.

8 Interview Corner

Q. What does ACID stand for?

A. Atomicity, Consistency, Isolation, Durability.

Q. What is MVCC?

A. Multi-version concurrency control: readers see a snapshot, so reads don't block writes.

Q. What is a dirty read?

A. Reading data from a transaction that has not committed (and may roll back).

Q. How do you prevent double booking?

A. SELECT ... FOR UPDATE, optimistic version check, or a unique constraint; or Serializable.

Q. Optimistic vs pessimistic locking?

A. Optimistic: rare conflicts, retry. Pessimistic: frequent conflicts, lock first.

⚡ Revise in 60 seconds

→ **Transaction**: all or nothing
→ **ACID**
→ WAL → A + D
→ MVCC / locks → I
→ Dirty · non-repeatable · phantom
→ RC (PG) · RR (MySQL) · Serializable
→ Write skew → double booking
→ FOR UPDATE vs version

Next lesson → SD-15
Caching deep dive: strategies, eviction and invalidation.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

Caching Deep Dive

Lesson 15 · strategies, eviction & invalidation

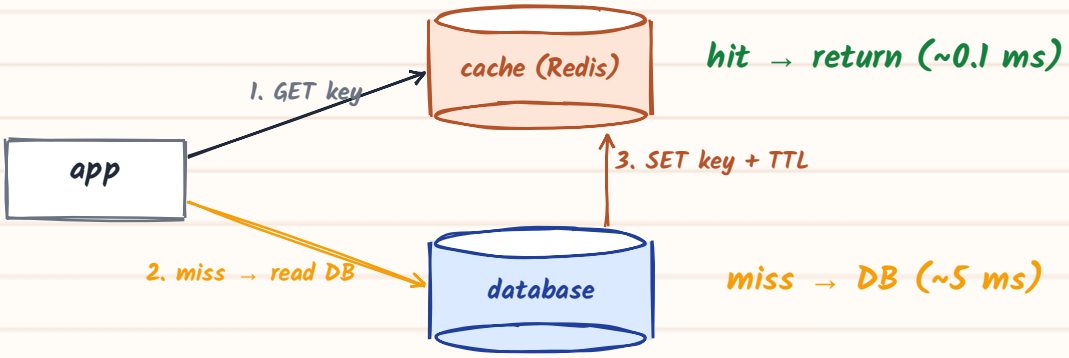
In this lesson → ① Why cache ② Cache-aside ③ Write strategies ④ Eviction ⑤ Invalidation ⑥ 3 disasters ⑦ Sizing ⑧ Interview Q&A

Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

> A **cache** keeps hot data in fast memory so most reads never reach the database. Redis ≈ **0.1 ms**, DB ≈ **5 ms**.

→ Layers: browser → CDN → in-app → Redis/Memcached → DB buffer pool

1 Cache-aside (lazy loading)



- ✓ Read cache → hit: return
- ✓ Miss: read DB → SET key with TTL
- ✓ Only data that is read gets cached

SYSTEM DESIGN NOTEBOOK

2 Write strategies

Strategy	On write	Good	Risk
Write-through	cache + DB together	always fresh	slower writes
Write-around	DB only	no churn	first read misses
Write-back	cache now, DB later	fastest	loss on crash

3 Eviction



LRU: cache full → evict the least recently used

LFU: evict the least frequently used (counts hits)

TTL: every key expires after N seconds anyway

→ **LRU**: default choice
→ **LFU**: protects long-term favourites
→ **TTL**: bounds staleness

4 Invalidation

- ✓ Update DB → **delete** the key (don't set it)
- ✓ TTL as a safety net
- ✓ Many caches: change events (CDC) → delete everywhere

5 3 cache disasters

Disaster	What happens	Fix
Stampede	hot key expires → herd hits DB	lock / single-flight, early refresh
Penetration	non-existent keys always miss	cache nulls, Bloom filter
Avalanche	many keys expire / cache down	TTL jitter, replicas, circuit breaker

6 Sizing + SnapLink

- → 80/20 rule: cache the hot ~20%
- → Target hit ratio 90%+
- → Redis Cluster: 16,384 hash slots, replicas

SnapLink
Cache feed pages, profiles and like counters (write-back for counters).

7 Interview Corner

Q. Cache-aside or write-through?

A. Cache-aside for read-heavy, stale-tolerant data; write-through when reads must be fresh.

Q. LRU or LFU?

A. LRU by default; LFU when popularity is long-lived.

Q. How do you invalidate?

A. Update the DB, then delete the key; TTL as backup; CDC events for many caches.

Q. What is a cache stampede?

A. A hot key expires and many requests hit the DB; fix with a lock or early refresh.

Q. How big should the cache be?

A. Enough for the hot set (~20% of data); tune by hit ratio.

⚡ Revise in 60 seconds

- → **Cache-aside** default
- → Write-through / around / back
- → **LRU** · LFU · TTL
- → DB → **delete** key
- → Stampede → lock
- → Penetration → nulls, Bloom
- → Avalanche → jitter
- → 80/20, hit ratio 90%+

Next lesson → SD-16
Object storage and big files: S3, chunking, pre-signed URLs.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**

Object Storage & Big Files

Lesson 16 · S3, pre-signed URLs & multipart upload

In this lesson → ① Photos in the DB? ② Block/file/object ③ Bytes vs metadata ④ Pre-signed URLs ⑤ Multipart ⑥ Pipeline ⑦ Durability + cost ⑧ Interview Q&A
Running project: SnapLink, our photo-sharing app that grows from 1 server to 100M users.

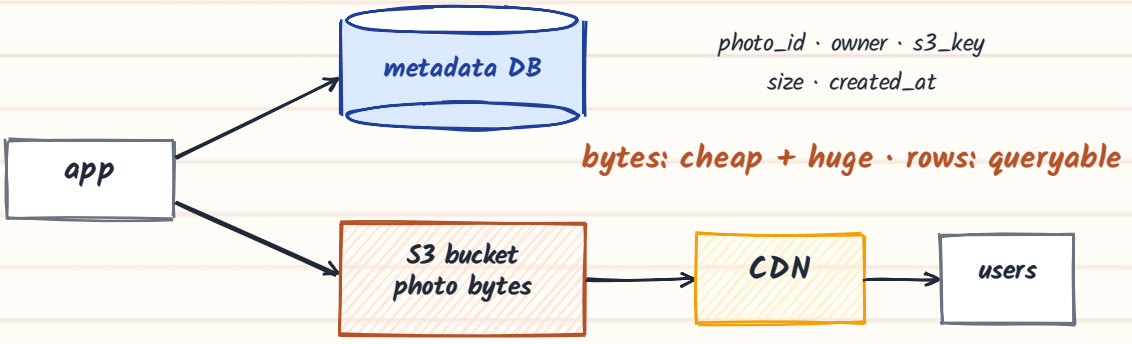
> **Object storage** (S3, GCS, Azure Blob) stores **objects** (bytes + metadata) in **buckets** under a **key**, over HTTP. Cheap, unlimited, 11 nines durability.

✗ Photos as DB blobs: huge DB, slow backups, every replica copies everything

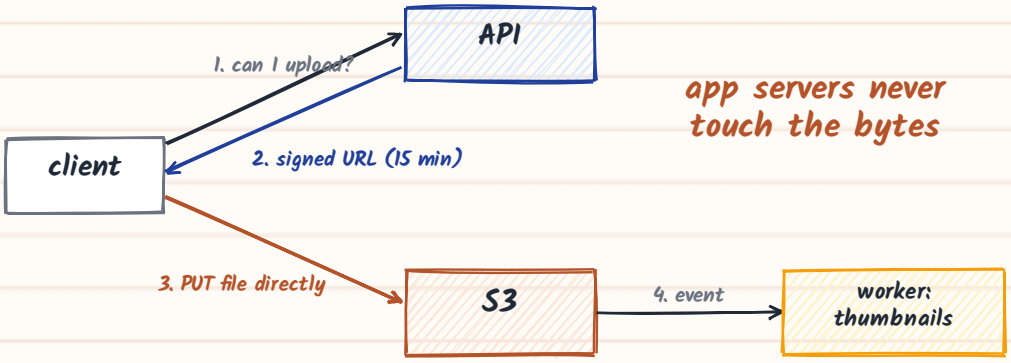
1 Block vs file vs object

Type	Looks like	Used for
Block	raw disk (EBS)	databases, VMs
File	shared folders (NFS, EFS)	shared files
Object	bucket + key → bytes	photos, video, backups, logs

2 Bytes in S3, metadata in the DB

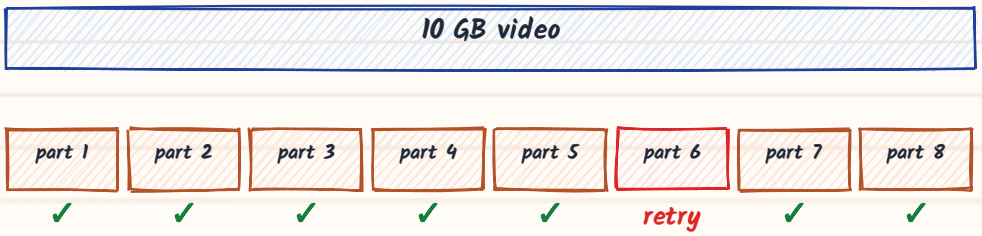


3 Pre-signed URLs



- ✓ API checks permission → returns signed, time-limited URL
- ✓ Client PUTs directly to S3 (also for downloads of private files)
- ✓ S3 event → worker (thumbnails)

4 Multipart upload



- → Parts (e.g. 100 MB) in parallel
- → Retry only failed parts; resumable
- → "Complete" call + checksums

5 Processing pipeline

→ Upload event → queue → workers

→ Thumbnails + sizes, strip EXIF, abuse scan, video transcode

→ Variants → bucket → served via CDN

6 Durability + cost

Class	For	Cost
Standard	hot, recent	$$$
Infrequent access	older than ~30 days	$$
Archive (Glacier)	backups, compliance	$

✓ Replication / erasure coding across zones

✓ Lifecycle rules move objects between classes

✓ Content-hash keys → dedupe

SnapLink
20 TB/day → S3 + CDN. Metadata row per photo in Postgres. Uploads via pre-signed URLs, thumbnails by workers.

7 Interview Corner

Q. Where do you store user images?

A. Object storage for bytes, DB for metadata, CDN for delivery.

Q. Why pre-signed URLs?

A. Direct client ↔ storage transfer; app servers stay free; access is time-limited.

Q. How do you upload a 10 GB file?

A. Multipart: parallel parts, retry failures, resume, then complete.

Q. How do you generate thumbnails?

A. Storage event → queue → worker writes variants back to the bucket.

Q. How do you reduce storage cost?

A. Storage classes, lifecycle rules, dedupe by content hash.

⚡ Revise in 60 seconds

- → Bytes → **S3**
- → Metadata → **DB**
- → Delivery → **CDN**
- → **Pre-signed** URLs
- → **Multipart** for big files
- → Async processing
- → Storage classes
- → 11 nines durability

Next lesson → SD-17
Search & special stores: inverted index, time-series, graph.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**